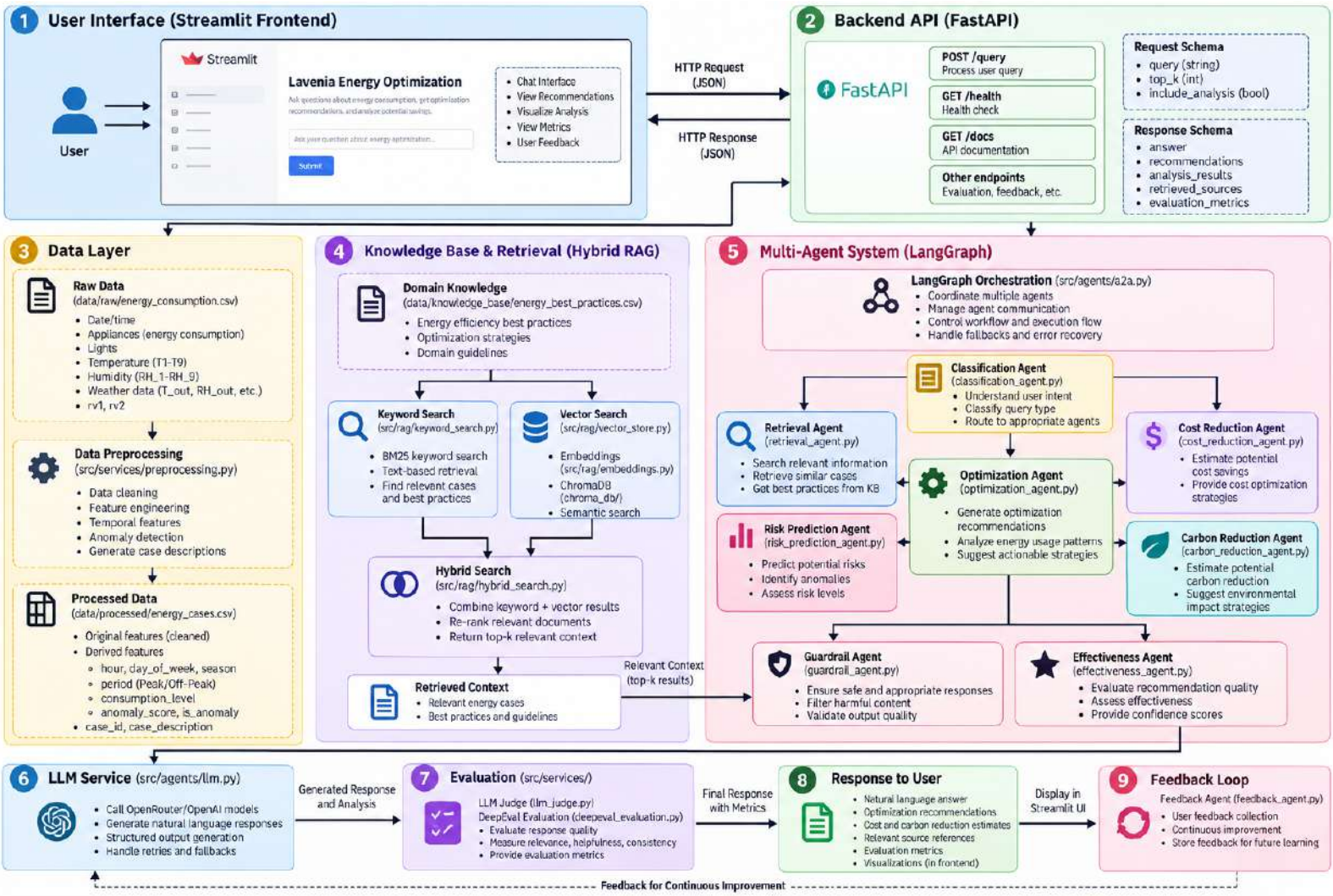

1 User Interface (Streamlit Frontend)
User
Streamlit
Lavenia Energy Optimization
Ask questions about energy consumption, get optimization recommendations, and analyze potential savings.
Ask your question about energy optimization...
Submit
Chat Interface
View Recommendations
Visualize Analysis
View Metrics
User Feedback
HTTP Request (JSON)
HTTP Response (JSON)
2 Backend API (FastAPI)
FastAPI
POST /query
Process user query
GET /health
Health check
GET /docs
API documentation
Other endpoints
Evaluation, feedback, etc.
Request Schema
query (string)
top_k (int)
include_analysis (bool)
Response Schema
answer
recommendations
analysis_results
retrieved_sources
evaluation_metrics
3 Data Layer
Raw Data
(data/raw/energy_consumption.csv)
Date/time
Appliances (energy consumption)
Lights
Temperature (T1-T9)
Humidity (RH_1-RH_9)
Weather data (T_out, RH_out, etc.)
rv1, rv2
Data Preprocessing
(src/services/preprocessing.py)
Data cleaning
Feature engineering
Temporal features
Anomaly detection
Generate case descriptions
Processed Data
(data/processed/energy_cases.csv)
Original features (cleaned)
Derived features
hour, day_of_week, season
period (Peak/Off-Peak)
consumption_level
anomaly_score, is_anomaly
case_id, case_description
4 Knowledge Base & Retrieval (Hybrid RAG)
Domain Knowledge
(data/knowledge_base/energy_best_practices.csv)
Energy efficiency best practices
Optimization strategies
Domain guidelines
Keyword Search
(src/rag/keyword_search.py)
BM25 keyword search
Text-based retrieval
Find relevant cases and best practices
Vector Search
(src/rag/vector_store.py)
Embeddings (src/rag/embeddings.py)
ChromaDB (chroma_db/)
Semantic search
Hybrid Search
(src/rag/hybrid_search.py)
Combine keyword + vector results
Re-rank relevant documents
Return top-k relevant context
Retrieved Context
Relevant energy cases
Best practices and guidelines
Relevant Context (top-k results)
5 Multi-Agent System (LangGraph)
LangGraph Orchestration (src/agents/a2a.py)
Coordinate multiple agents
Manage agent communication
Control workflow and execution flow
Handle fallbacks and error recovery
Classification Agent
(classification_agent.py)
Understand user intent
Classify query type
Route to appropriate agents
Retrieval Agent
(retrieval_agent.py)
Search relevant information
Retrieve similar cases
Get best practices from KB
Optimization Agent
(optimization_agent.py)
Generate optimization recommendations
Analyze energy usage patterns
Suggest actionable strategies
Cost Reduction Agent
(cost_reduction_agent.py)
Estimate potential cost savings
Provide cost optimization strategies
Risk Prediction Agent
(risk_prediction_agent.py)
Predict potential risks
Identify anomalies
Assess risk levels
Carbon Reduction Agent
(carbon_reduction_agent.py)
Estimate potential carbon reduction
Suggest environmental impact strategies
Guardrail Agent
(guardrail_agent.py)
Ensure safe and appropriate responses
Filter harmful content
Validate output quality
Effectiveness Agent
(effectiveness_agent.py)
Evaluate recommendation quality
Assess effectiveness
Provide confidence scores
6 LLM Service (src/agents/llm.py)
Call OpenRouter/OpenAI models
Generate natural language responses
Structured output generation
Handle retries and fallbacks
Generated Response and Analysis
7 Evaluation (src/services/)
LLM Judge (llm_judge.py)
DeepEval Evaluation (deepeval_evaluation.py)
Evaluate response quality
Measure relevance, helpfulness, consistency
Provide evaluation metrics
Final Response with Metrics
8 Response to User
Natural language answer
Optimization recommendations
Cost and carbon reduction estimates
Relevant source references
Evaluation metrics
Visualizations (in frontend)
Display in Streamlit UI
9 Feedback Loop
Feedback Agent (feedback_agent.py)
User feedback collection
Continuous improvement
Store feedback for future learning
Feedback for Continuous Improvement